Technology Feasibility

CardioCare Quest

Project Sponsor: Dr. Jared Duval, Assistant Professor

Mentor: Morgan Nicholson

Playful Health Technology Lab

Members:

Jesse Ceja

Tyler Jeffrey

Rique Yazzie

Anna Cheatham

03/05/2026

Table of Contents

Introduction

Context

Hypertension is globally estimated to impact 1.4 billion adults, yet only 25% of that population has the condition under control. This issue disproportionately affects Indigenous communities, facing prevalence rates 10-20% higher than non-Indigenous groups. These disparities emerge as a direct manifestation of systemic technological and healthcare inequalities alongside a severe cultural mismatch in medical interventions. When tools designed to save lives fail to speak the language of the people they serve, the result is a widening gap in health equity and a preventable loss of life.

The Problem

Current hypertension management relies on clinical tracking apps and pharmaceutical regimes that feel isolating and transactional. These solutions adopt a one-size-fits-all approach that promotes a sterile, data-centric relationship between patients and their own health as a result of ignoring the social and cultural dimensions of medicine. Our client, Dr. Jared Duval, is the director of the Playful Health Technology (PHT) Lab that primarily studies human-computer interaction technologies in order to enhance and maintain health within underserved populations. His work has led to a series of game technologies dedicated to dismantling these clinical barriers by engaging users in medically relevant forms of play.

However, the process of developing new medical technologies for Indigenous communities has reached a bottleneck. Currently, managing heart health within these regions involves a difficult process of trying to superimpose medical models onto Indigenous contexts. This results in low engagement rates and a profound lack of trust within patients. The current “solution” is to use Twine, a game creation tool, allowing members of the community to create and import their own narratives to boost engagement and trust. This system alone, however, is inadequate because such tools can be too complex or code heavy for non-technical users. It creates a fragmented system where the technology itself, much like existing medical apps, acts as a barrier rather than a bridge, preventing the exchange needed to make culturally grounded medical interventions successful.

Goal

In response, we are developing **CardioCare Quest**, a mobile health gaming environment meant to replace standard health tracking applications with a culturally grounded form of play within the Navajo community. This app is a platform that allows users to track hypertension data by engaging in Twine narratives that reflect the community’s values and traditions.

Our role in this project is split into two core technical and creative responsibilities:

- **The Unified App Shell:** We are designing a seamless experience that allows users to navigate between the main app and various sub-games with ease. This shell will be based on the PHT Lab's other application, *NetGauge*, to ensure a consistent visual style between projects.

- **The Exemplar Games:** We will be creating 3-5 specialized Twine games inspired by workshops conducted within the community, each focused on collecting a specific hypertension metric (such as dietary logs or blood pressure). These games will serve as examples, providing a blueprint for how future community-generated content can utilize storytelling as a form of medical data collection. This also includes extensive documentation and organization within our games to ensure non-technical users are able to produce their own Twine narratives.

By integrating these features, CardioCare Quest hopes to build a strong base for the future of medical data collection whilst also having broader implications in the world of culturally responsive digital health innovations.

Now that CardioCare Quest's importance has been established, the rest of this document will turn to the technological feasibility analysis of this project's development. At this early stage, we are identifying key technological challenges, analyzing possible alternatives, and selecting the most promising solution to each challenge. This is done to move away from a conceptual vision into building a successful foundation for our app.

Technological Challenges

In order to actualize CardioCare Quest, several key technical hurdles must be cleared. These challenges represent core design decisions that have a strong impact on determining whether the platform succeeds as a health tool. As such, we must balance the PHT Lab's vision for our app while also ensuring our product remains accessible, high performing, and effective.

1. **Culturally Grounded & De-Clinicalized Interface:** The primary challenge for the user experience is removing the "sterile" feel common in health trackers. We need a way to provide a reactive and immersive interface that prioritizes Indigenous aesthetic values over data-heavy visualizations. This requires input from workshops conducted within the community by the PHT Lab as well as cross-referencing designs between related projects.

2. **Twine to Dart/Firebase Bridge:** One of the most prominent hurdles of this project is the data collection process. We need a reliable way to enable Twine games to communicate with a developed Dart backend in order to transmit hypertension related data into a Firebase library. This involves establishing a stable Twine to Dart bridge, allowing us to utilize the data collection framework provided by the PHT Lab with JavaScript as the primary language for communication.

3. **Creating Human Readable Documentation:** Because community generated content is such a core aspect of our product, it is very important for us to overcome the technical nature of Twine. For this, we face the challenge of producing a very human-readable documentation of our Twine games that translates abstract concepts into easy to understand explanations. This also extends to the organization and presentation of ideas within the game engine to ensure they are easily accessible for auditing and editing.

4. **Cross-Platform Compatibility:** As a mobile application, CardioCare Quest aims to function well across Android and iOS devices. To this end, we require a cross-platform development tool that ensures full compatibility of all features within our app while also minimizing the development time required designing platform specific implementations.

Challenge 1: Culturally Grounded & De-Clinicalized Interface

Issue:

Current Health apps often only present ideals that are mainstream assumptions. As a result, people of differing cultures often feel isolated by the health apps offered to them. To remedy this, our client Dr. Duval helped to lead a community workshop within the target audience's community aimed at co-designing an application made for them. Within this workshop, multiple key attributes were identified.

Desired Characteristics:

1. **Live Data Collection and Presentation**

A quality that had much praise within the workshop was a "Point System" that offered immediate positive feedback to the user, as well as offering a communal score. To that end, our technology needs to have a robust method of rapidly gathering data and presenting it to the user.

2. **High Color Range**

A pattern that arose was a wide variety of available colors. Ranging from soft-color palettes for a relaxed experience to vibrant visuals allowing for thrilling experiences, the chosen technology must enable the creation of dynamic textures.

3. **Unique Input Methods**

In the workshop, designs that shined featured input methods ranging from the simple, irregular shaped buttons, to the extreme, augmented reality experiences. To achieve this, our technology needs to be suited to building new input methods as needed.

Alternatives:

1. **Flutter Framework**
   Flutter is a SDK developed by Google specifically for compiling on native devices, including embedded devices. It uses the Dart Programming language, also developed by Google, which is open source. Thanks to this, many public libraries exist for us to use.
2. **Android Studio**
   An IDE also developed by Google, but specifically aimed at Android development. This includes the range of Android devices, including tablets, TV's Wearable devices, and Automotives. It is incredibly powerful within this ecosystem. Cross-platform exists via Kotlin Multiplatform.
3. **Godot**
   A dedicated game engine that also has mobile application support. It is incredible for presenting a wide range of visuals and live data. Due to it being open source, it is very well documented and has libraries built for connecting with existing 3rd party technologies.

Analysis: (1/5 Scale, 1 being the lowest)

These judgements were based on personal experience building apps within Godot and Android Studio ranging from simple tech demos to full scale projects. For testing Flutter I created a very simplistic app to measure its capabilities, as well as viewing an existing codebase built by our client, the following rankings were found.

Alternatives	Live Data	High Color Range	Unique Input Methods
Flutter	**4**	**4**	**4**
Android Studio (Kotlin)	**4**	**2**	**3**
Godot	**4**	**5**	**2**

Chosen Approach:

After testing Flutter, it was found to be the most accommodating to our desired characteristics. While Godot is fantastic for any number of graphics, it is limited by its “scene” style structure having a limited amount of input methods. Android Studio was found to be well suited for developing within Android, however the process of designing with a wide range of colors proved tedious, and would fail at the scale we are aiming for. Flutter was able to meet our expectations and was relatively easy to do so. Thanks to this ease of creating design, we decided Flutter would be the best suited for creating designs that meet all criteria.

Proving Feasibility:

In order to prove Flutter is the optimal technology, a demo we plan on developing is a simple app that offers multiple navigational options to reach pages within the app that display the color range and live data discussed. The navigational options will show the variety of ways the app can be used, and the live data will take user input in a multitude of ways to show the availability in that regard. This tech demo will be developed later as part of an upcoming deliverable.

Challenge 2: Twine to Dart/Firebase Bridge

Issue:

Twine is a web-based game engine that runs inside of a browser environment and on its own does not have direct access to native mobile device features. When integrating Twine games into a Flutter application it can create limitations where the game cannot independently access functions like GPS location, or backend storage like Firebase can. Keeping this in mind we need to create a communication mechanism to help bridge the gap between the JavaScript running in the Twine game and the native Flutter application,

Desired Characteristics:

1. **Two-way data communication capabilities**

The bridge should help allow Twine games to send structured messages to the Flutter application and receive responses for various functionality. This would ensure smooth interaction between the game logic and native services

2. **Support for Native Feature Access**

The bridge created should allow for Twine games to use device features like location or vibrations that are not available inside of a normal web environment.

3. **Efficient Data Handling and Storage**

The bridge should be able to send data payloads like player actions, session data, and other forms of data from the game to Flutter for processing and storage inside of backend systems like Firebase.

Alternatives:

1. **SugarCube**
   A Twine story format that excels at handling complex logic, variables, and state management, making it well-suited for games with inventory systems or complex story branches. It features a robust macro set, built-in Save/Load functionality, and support for complex JavaScript scripting
2. **Harlowe**
   Harlowe is designed for creating interactive fiction without requiring knowledge of HTML, CSS, or JavaScript. It uses a simple macro-based syntax for links, variables, and styling, making it ideal for text-heavy, branching stories.
3. **Snowman**
   A minimal, code-heavy Twine story format designed for users more comfortable with JavaScript and CSS. It includes libraries like jQuery, Underscore.js, and Marked by default, allowing for high customization with very little built-in structure.

Analysis: (1/5 Scale, 1 being the lowest)

Alternatives	Two-way data communication	Support for Native Feature Access	Efficient Data Handling and Storage
SugarCube	**5**	**5**	**5**
Harlowe	**2**	**2**	**2**
Snowman	**5**	**3**	**3**

Chosen Approach:

Harlowe is considered the least suitable story format due to its lower scores in all categories. Its limited JavaScript support highly restricts two-way data communication for messages to be received outside the Twine environment. It is more limited in supporting native feature accesses and its simplified variable system reduces its effectiveness in efficient data handling and storage, especially when compared to other more flexible story formats. Snowman did score significantly higher than Harlowe as it allowed full JavaScript control, enabling strong two-way data communication and higher customization options. But what sets it apart from SugarCube was its features regarding native feature access and data handling features as it does not include built-in systems for state management. This requires developers to manually implement data structures and storage logic, increasing development complexity and reducing overall efficiency.

SugarCube was chosen as the preferred story format due to its strong support for JavaScript and its ability to handle complex game logic. This enables reliable two-way communication, allowing for easy implementation of structured messaging that happens between the Twine games and the Flutter application through the bridge. Unlike Harlowe or Snowman, SugarCube provides vast built-in systems and has flexible scripting capabilities that make it easier to implement and manage communication with no extra work. Overall, SugarCube was found to provide the best balance between its functionality and ease of integration with its native features.

Proving Feasibility:

To demonstrate that SugarCube is the optimal Twine story format, a demo we plan to develop is showcasing the functionality of how SugarCube can handle communication with Flutter applications through the bridge using JavaScript. The demo will include simple message parsing as well as sending structured payloads of game data such as player actions or session information formatted as JSON to be transmitted to Flutter for processing and storage.

Challenge 3: Creating Human Readable Documentation

Issue:

With this project, it is very important that we keep our code base clean and understandable. The plan for the future is for the Navajo people to be able to make their own games. We want to be able to give everybody an equal opportunity to create these games, despite their technological backgrounds. We don't want our twine games to be unreadable and confusing, so they may easily be able to follow along with what the code is doing.

Desired Characteristics:

1. **The source of documentation needs to be accessible.**

Wherever the documentation lies, it should be easy to access without going through a lot of steps to get there. The information needs to be readily available so users have an easier time using that documentation and code to learn and make their own games.

2. **Documentation needs to be easy to read**

It is important that the documentation is as simple as it can be. It should outline everything that the code is doing, without sounding too complex or confusing with fancy vocabulary. It also shouldn't be too long in length as there is no need to read large paragraphs for a small chunk of code when a simple sentence does the trick.

3. **Users' don't need a past with making games to understand**

It is not expected for people to have past programming experience to be able to add games to CardioCare Quest. It is important that anyone can pick it up, so the documentation should provide all the information someone would need to understand what the code is doing so they could replicate it in their own game. The documentation should almost serve as a learning opportunity rather than a quick explanation for other programmers.

4. **Documentation should be cleanly organized**

Even if documentation is readable and easy to understand, if it is all over the place, it will still be confusing. Things should be neatly organized in a way that the code and the steps it takes are easy to follow. Users who are reading it shouldn't have to really search for something, rather it should be a quick find.

Alternatives:

1. **Rules and Guidelines for Documenting in Twine**
   This is how most programs are written with commenting done inside the code. Creating rules the team will follow will allow for more consistency across the code base.
2. **A Google Doc giving more comprehensive documentation**
   Google docs can be a good alternative that can provide a more thorough explanation into what the code was doing without flooding the code base with comments.

Analysis: (1/5 Scale, 1 being the lowest)

These ratings are based on personal experience documenting in both Twine and using Google Docs. I had made a simple program in twine and documented it the best I could. Then, I used Google Docs to document that program I created.

Alternatives	**Easy to Read**	**No Past Experience Needed**	**Organization Capabilities**	**Accesibilty**
In Code Base with Guidelines	**4**	**3**	**4**	**5**
Google Docs	**4**	**4**	**5**	**3**

Chosen Approach:

When it comes to choosing which one to go with, the analysis is pretty split, as they average the same score. However, the documentation being easily accessible is the most important characteristic of this as outlined by Dr. Duval, so creating guidelines for documentation inside Twine is the approach we will be using. It has some faults with experience needed, but if the documentation is done well, then that shouldn't cause too much of an issue in the end.

Proving Feasibility:

To prove that documentation inside the Twine code base is the best approach, we will be creating a simple game and use guidelines we come up with to document it thoroughly. We will provide annotated screenshots of how the code is organized and documented and show that it is easy to understand and follow.

Challenge 4: Cross Platform Compatibility (Android & iOS)

Issue:

This challenge primarily lies within the architectural divergence between Android and iOS platforms. For our project to succeed, the app must provide a unified user experience across the two hardware capabilities and operating system permissions.

Desired Characteristics:

1. **Development Velocity**

The framework should maximize the amount of shared code, preventing redundant work a s well as enabling features to be shipped across both platforms without having our design driven by the limitations of a single platform.

2. **Access to Native APIs**

The chosen framework should interact well with OS related features such as local file storage and haptic feedback. This is important in the areas of hardware access, performance, and within platform-specific features between Android and iOS.

3. **Firebase SDK Correspondence**

We require a unified Firebase library between iOS and Android. This framework should provide a 1:1 synchronicity between each SDK to prevent potential data divergence.

Alternatives:

1. **Flutter**
   Flutter has been identified across a variety of developer forums as well as a recommendation from our client, cited primarily for its UI capabilities. Developed by Google, Flutter is an open-source UI software development kit (SDK) using the Dart programming language. It provides its own framework for working between Android and iOS.
2. **React Native**
   React Native, developed by Meta, is widely used for cross-platform apps and is recommended in forums from a technical standpoint for having a singular codebase. This framework allows us to utilize React and JavaScript to build mobile applications and bridge native UI components between each platform.
3. **Kotlin Multiplatform (KMP)**
   This was identified through developer blogs and research into popular cross-platform technologies. Kotlin Multiplatform is a Kotlin-based framework that allows code to be reused across a variety of platforms.

Analysis: (1/5 Scale, 1 being the lowest)

Evaluation for these frameworks were conducted based on a process of comparative analysis and research into the three main desired characteristics. This investigation is primarily focused on publications about these technologies, plugins for Firebase integration, and the availability of APIs potentially required for this project:

Alternatives	**Development Velocity**	**Native API Access**	**Firebase Correspondence**
Flutter	**5**	**3**	**5**
React Native	**4**	**4**	**4**
Kotlin Multiplatform	**2**	**5**	**3**

Chosen Approach:

Our investigation reveals **Flutter** to be the strongest candidate. The pipeline between Firebase and Flutter is seamless due to the “FlutterFire” tool also managed by Google, which is the most comprehensive and stable implementation of Firebase. Additionally, Flutter provides very reliable cross-platform capabilities via the Impeller engine allowing us to draw our own UI independent of iOS, Android, or web platforms. In terms of code-reusability and development velocity, teams have reported a **30-40% increase** compared to native development.

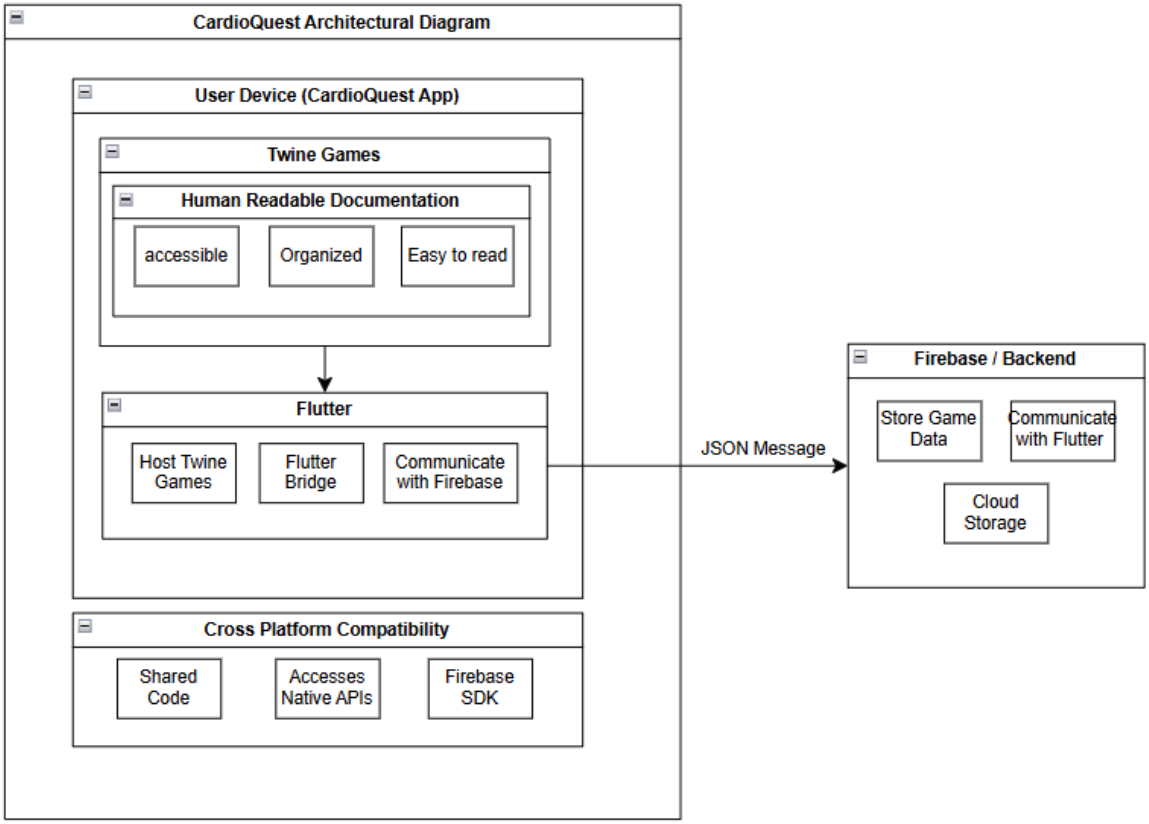
Kotlin Multiplatform and **React Native** emerge as reliable frameworks that could potentially be used in the place of Flutter due to performance and integration being similar. However, KMP is bottlenecked by challenges when implementing platform specific features and a steep learning curve because its iOS support is still maturing. React Native has a similar ecosystem, yet is more reliant on third-party dependencies when considering Firebase and other APIs which makes it more prone to risks of version instability and/or platform drift compared to Flutter.

Proving Feasibility:

To prove Flutter as the best framework for cross-platform development, we plan to create a demo showcasing a basic ability to create reusable code on each platform as well as integrating an example of the “FlutterFire” usage.

Technological Integration

This document provides a roadmap addressing multiple technical challenges including creating an interactive non-clinical application, creating a bridge between Twine and Flutter, providing beginner friendly readable documentation, and cross platform compatibility. These solutions come together to ensure CardioCare Quest successfully delivers seamless interaction, reliable data handling, and effectiveness for the users. The next step will be to integrate these solutions into a unified prototype application that demonstrates the technical feasibility of our approach. The integration will demonstrate that the created architecture supports communication between the Twine games and Flutter, access to native device features, cross-platform functionality, and user friendly content-creation.



Conclusion

Thanks to the development of mobile devices and the proliferation of their ownership, we have an opportunity to use these devices to bridge the gap within a system that has historically failed to make medical interventions successful. As current solutions ignore cultures that fall outside of the mainstream assumptions, distrust has been built between the medical community and Indigenous communities. The project we are undertaking, CardioCare Quest, is attempting to remedy this failure, and rise to the opportunity presented. The contents of this document serve as the proof of our ability to plan for success in this undertaking.

We have conducted thorough in-house testing as well as gathering research and exposure regarding a set of technologies we believe could hold merit in achieving our mission. Great help has come from Community Workshops held in co-leadership between our client Dr. Duval and leaders of the target demographic that have defined our measurement of each technology. It was with these defined measurements that we found the Flutter SDK to best match the design expectations created by the Workshops. We have also defined measurements of our own regarding the technologies that have been chosen for us by our client Dr. Duval, such as Twine. Largely, we wanted to ensure that the Twine Version we have chosen could handle all of the tasks we could throw at it. We have also found that documenting programming concepts and functions within the Twine games will be the best way to help others trying to develop their own Twine games as it simplifies the learning process. Finally, Flutter SDK was also identified as the most accommodating technology identified for Cross-Platform Compatibility, ensuring users of all devices can access our work.

It was with the aforementioned research and experience that we have found that overall the technologies:

- Flutter SDK
- Twine Sugarcube 2.37.3

Will prove the best suited to achieve our mission in bridging the gap. In order to assure this, we will supplement our current research and experience with further testing. We will be developing technology demonstrations for each of the chosen approaches that prove they can handle their desired characteristics.